

Stock Price Predictions

Yamato Matsumura
CSCI413

March 2025

1 Abstract

Predicting stock prices using Machine Learning (ML) has become an increasingly popular trend as the field of Artificial Intelligence continues to grow. However, these projects are often too theoretical and don't translate well to real applications. Thus, this research project aims to not only apply ML techniques to predicting stock prices, but also simulate market conditions.

To do this, data was fetched from the Tiingo API and feature engineered to add key indicators. This was then fed into a Long Short Term Memory Model, which predicted the next 30 days' worth of closing prices. Although the predictions varied, being very accurate at times while being very inaccurate at others, the model simulation yielded significant results.

Over the course of a month, the model achieved a 15% increase in portfolio value. Compared to the growth of common index funds like the S&P 500 and the QQQ fund, the model outperformed them by around 3% and 5% respectively over the same month.

Thus, the results from the research project indicate that modern approaches to stock price predictions using ML algorithms are entirely viable. With compute power being the most significant obstacle in this research project, further improvements could likely be achieved through access to more advanced hardware. Overall, these findings support the potential of ML as a competitive tool in financial forecasting and portfolio management.

2 Introduction

Although movies like *The Wolf of Wall Street* and *The Big Short* don't exactly represent realistic expectations when it comes to the stock market, they give valuable insights into a common goal some finance and business professionals have: beating the market and generating profits. This goal has existed for decades, with Benjamin Graham's "*The Intelligent Investor*" (1949) serving as a foundational piece of literature that shaped value investing and traditional stock market strategies. However, as markets have grown more complex, existing methods are becoming increasingly inefficient for consistently predicting stock movements. With the recent rise of Artificial Intelligence (AI) and Machine Learning (ML), however, new, more sophisticated techniques have been developed that allow investors to fully utilize the vast amount of data, recognize market trends, and make more accurate predictions.

3 Current Methods

Current applications of ML to stock price predictions generally fall into three categories: traditional machine learning models, deep learning architectures, and sentiment analysis techniques, each offering unique advantages.

Traditional machine learning mainly applies statistical methods to historical stock data to identify patterns for predictions. One common example is linear regression models. However, this approach assumes that “the input and output variables have a linear relation” which oversimplifies the market’s complex and volatile nature [1]. In Sangeetha’s study, although the linear regression model proved to be somewhat accurate, the graph comparing actual and predicted values shows how much error the model had [1].

Another traditional machine learning method is random forests, where multiple decision trees are combined to improve accuracy and reduce overfitting. However, like linear regression, this approach struggles to capture the complexity of stock price movements. In Dev’s study, random forest predictions were about \$150 off on average, with Dev noting that “stock price predictions are inherently uncertain” [2]. While both methods highlight the difficulty in predicting stock movement, they show how these new AI and ML approaches could potentially outperform traditional ones.

Due to the limitations of traditional ML approaches, deep learning architectures like Long Short-Term Memory (LSTM) models were developed to better capture complex, non-linear patterns in stock prices. This model uses a series of cells with specialized gates that allow the model to control the flow of information, allowing the model to learn the context of previous data points [3]. In Gulmez’s study, the LSTM model was significantly better than traditional methods, only averaging about \$5-\$10 of error for some stocks, compared to the \$150 average error from random forest [4].

Another AI/ML approach is sentiment analysis, which classifies large amounts of text data as positive, negative, or neutral [5]. In stocks, it’s used to gauge market sentiment, as seen in Palomo’s study using Twitter to classify tweets as bullish, bearish, or neutral [6]. Although this helps predict stock direction, it doesn’t help with specific price forecasts. Thus, it is often combined with models like the LSTM, as shown in Ouf’s study, which achieved reasonably accurate predictions using both methods [7].

However, these current methods have a significant flaw in that they are never actually tested on real market conditions. Instead of the model actually predicting prices, it simply becomes “really good at using previous day’s price to act as a stand-in for the next day’s price” [8]. Thus, this research paper aims to actually apply these predictions in simulated market conditions, and compare the performance of the model to more traditional methods of stock investment.

4 Data Collection and Preprocessing

All of the data used for this research project was pulled using the Tiingo API, which allowed simple access to the open, high, low, close, and volume (OHLCV) statistics. In terms of date ranges, Tiingo gave access to a significant amount of data, often only being limited by the stock not existing yet. Thus, stocks like Apple had around 40 years worth of data to use, totaling to around 11,000 observations. Since only the Tiingo API was used, the data for each stock was clean and did not require additional wrangling to format. However, this was also one of the major challenges for this project. Since multiple stocks would be tracked, any API used would need to have very high API call limits to allow

smooth data collection. Thus, although other APIs like Alpha Vantage and Polygon API featured more rich data, they were unable to be used due to call limits.

Since only OHLCV data was pulled, feature engineering was implemented to add additional features to the dataset.

Category	Features
Original Price Data	date, close, high, low, open, volume
Moving Averages	sma_50, sma_200, ema_50, ema_200
RSI Indicators	rsi, rsi_overbought, rsi_oversold
MACD Indicators	macd, macd_signal, macd_histogram
Volatility Measures	volatility_10d, volatility_30d
Rate of Change	roc_10, roc_30
Date Features	Sin Date, Cos Date
Other	dist_from_ema_200

Table 1: List of Features included in the Dataset

Table 1 outlines these added features, totaling to 23 features. Notably, the added features help encode helpful statistics over different windows, like “sma_50” representing the simple moving average over a 50 day window and “ema_200” representing the exponential moving average over a 200 day window. This helps the model easily recognize long term patterns over the dataset, with the overall goal of playing to the strengths of the LSTM model.

Following this, the data was normalized using Z-Score normalization, ensuring the model wouldn’t be biased towards features with higher values numerically. Next, the data was windowed, allowing the model to get an input of 30 days worth of data, and output the next 30 days worth of closing prices. This also had the overall goal of helping the LSTM model learn patterns over numerous days, as opposed to only being able to input it one set of features, and it output one predicted closing price.

Finally, the data was split into training and testing subsets, where only the last windowed portion of data was kept for testing. This allowed the model to learn the newest available data along with the goal of simulating real time stock conditions, where the accuracy entirely depends on one set of outputs from the model.

4.1 Ethical Considerations

In this project, the primary ethical considerations arose during the data collection stage, particularly concerning a key data privacy issue: insider trading. Insider trading involves corporate employees exploiting confidential, private information about their own company to gain an unfair advantage in the stock market. This is usually in the form of buying their own company stock when they get access to private information that their company stock may rise. To put this into perspective, Jaffe’s study found that within 45 different companies, when insiders would buy their own company stock and hold onto it for roughly six months, their returns would be around 10% greater than average returns based on the stock market [9]. From an outsiders perspective, attempting to gain access to these sorts of private information would be against data privacy laws. Thus, for this project, only publicly available data sources like APIs were utilized even if insider information would most likely improve the results of the project.

5 Exploratory Data Analysis

To help explore the dataset and for visual clarity, the majority of the observations in the dataset were removed, only keeping about the last three years worth of data. In addition to this, Apple stock data was chosen for the following figures for simplicities sake. However, similar trends to those discussed where also seen in other stocks as well.

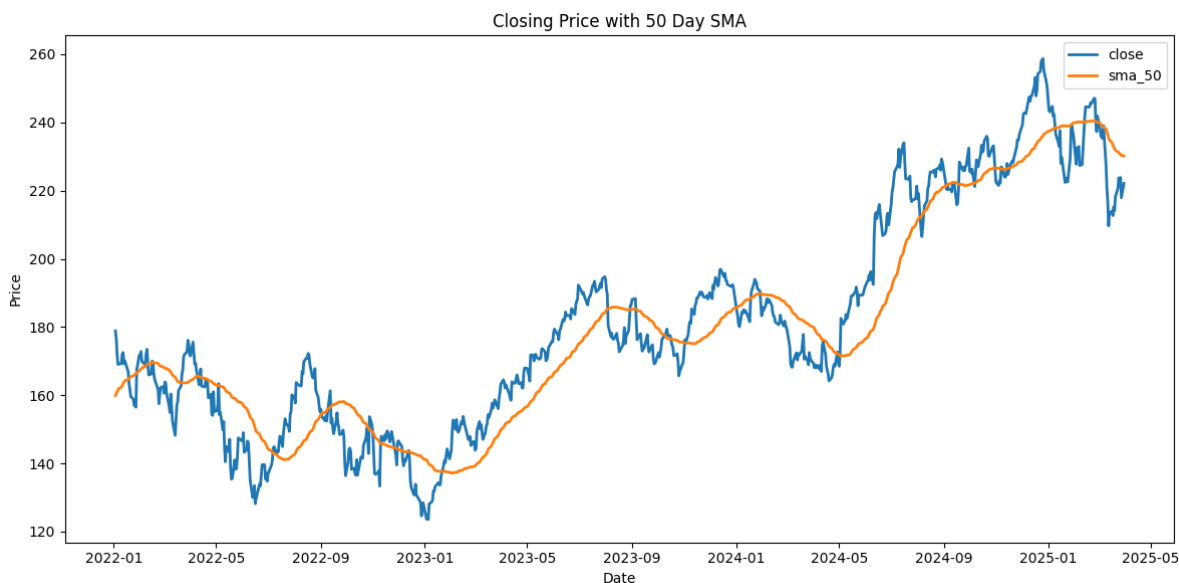


Figure 1: Line Graph Showing the Relationship Between Closing Price and SMA

Figure 1 gives insights into the effects and goals of the feature engineering. As seen in the figure, the Simple Moving Average (SMA) over a 50 day window essentially helps smooth out the closing prices curve. Thus, this added feature serves the goal of helping the LSTM model generalize to the stock movement, helping it not overfit on the noise or spikes present in closing prices over time.

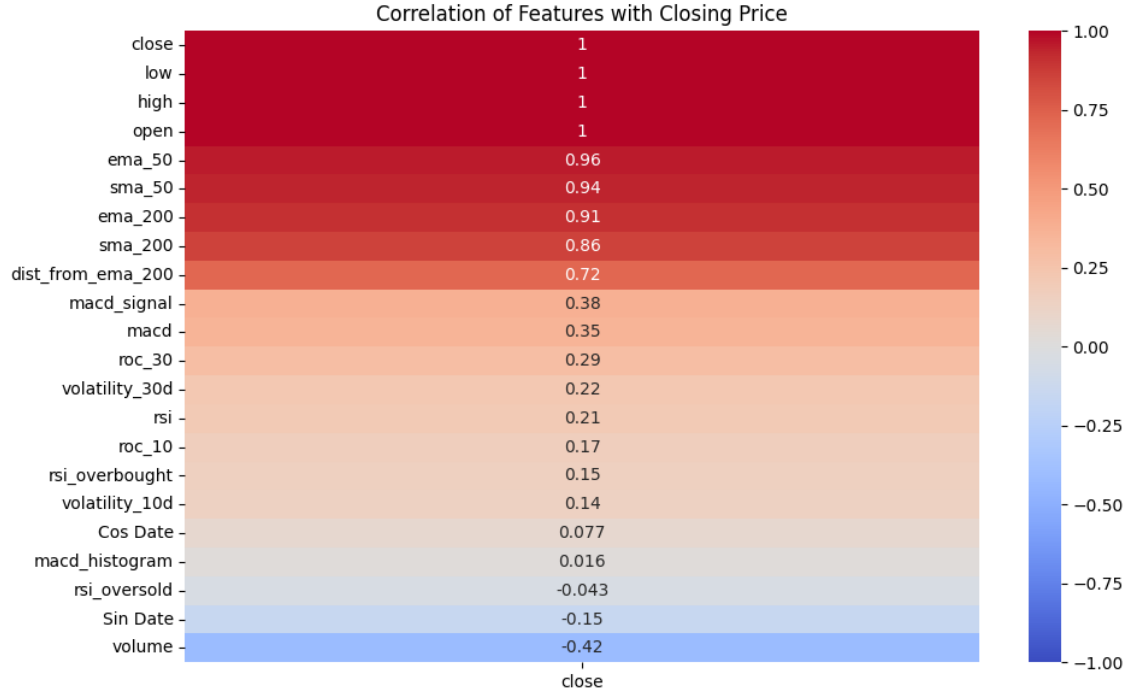


Figure 2: Heatmap showing Correlation with Closing Price

Figure 2 shows some interesting observations from the dataset. Trivially, we see that the open, high, and low statistics are all highly correlated with the closing price. This makes sense as is the nature of stock data. However, one interesting observation from this is the slight negative correlation of volume with closing price. This already shows a common trend in stocks, where lower prices usually results in higher volumes purchased, leading to increased stock prices, resulting in fewer volumes purchased. Thus, these correlations present in the dataset may help the model pick up on the inherent patterns that exist in stock data.

	open	high	low	close	volume
mean	180.657132	182.601741	178.876942	180.840156	6.693858e+07
std	31.974918	32.078175	31.847285	32.014337	2.794618e+07
min	124.565528	126.305353	122.746620	123.586876	2.323470e+07
25%	156.400882	158.437941	153.497826	155.997406	4.801327e+07
50%	173.878259	175.688326	172.283168	173.742521	6.075025e+07
75%	195.725670	196.819110	194.218843	196.391674	7.947097e+07
max	257.906773	259.814677	257.347387	258.735862	3.186799e+08

Figure 3: Statistical Summary for Key Features

Finally, we see a statistical summary for our key features in Figure 3. We see a significant range in values when comparing volume to the other key features. Thus, this helps highlight the necessity in data normalization to ensure the model is able to not become biased on certain features.

6 Model Development

For this project, two main models were applied: Linear Regression and LSTM. Notable results of the models can be found below.

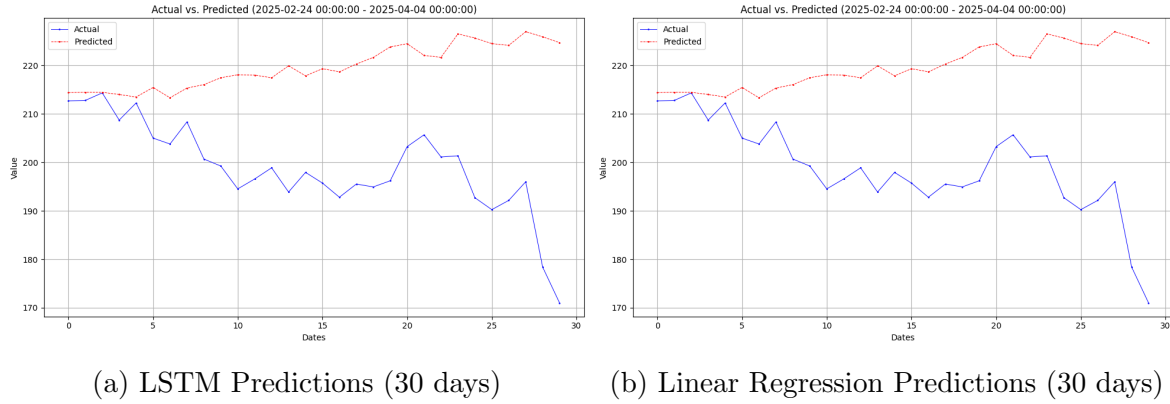


Figure 4: Comparison of AMZN Closing Price Predictions between the LSTM Model and the Linear Regression Model (30 days)

Metric	LSTM	Linear Regression
RMSE	24.334	23.359
R Value	-0.67	-0.611

Table 2: Comparison of AMZN Stock Model Evaluation Metrics for LSTM vs. Linear Regression

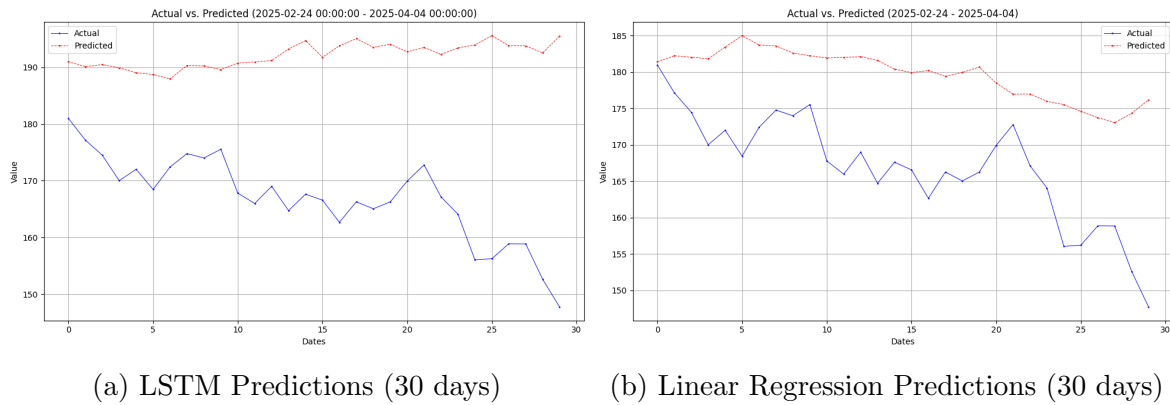


Figure 5: Comparison of GOOG Closing Price Predictions between the LSTM Model and the Linear Regression Model (30 days)

Metric	LSTM	Linear Regression
RMSE	26.764	13.985
R Value	-0.677	0.73

Table 3: Comparison of GOOG Stock Model Evaluation Metrics for LSTM vs. Linear Regression

Based on these results, we see that both models are similar in their performance. This was a common pattern found throughout tests of different stocks, where sometimes the LSTM would be better than the Linear Regression, and vice versa. However, due to the greater complexity and expected performance of the LSTM model, it was chosen to be used in the simulation over the linear regression model.

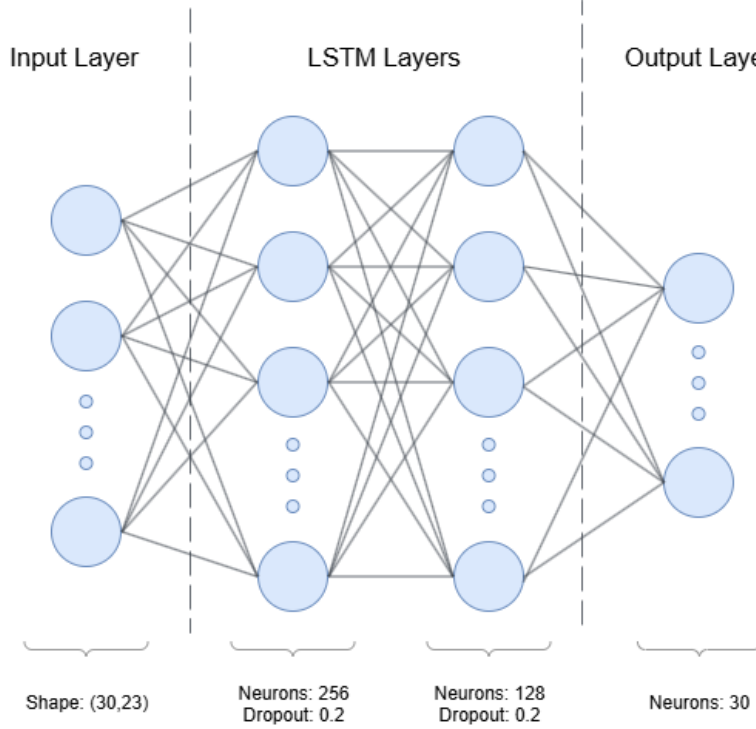


Figure 6: Visual Representation of LSTM Model

Figure 6 shows the final LSTM network that was used for the project. It consists of an input layer of 30 observations, each with 23 features. This would go through two LSTM layers of 256 and 128 neurons respectively, each with 0.2 dropout. This would then output the next 30 days worth of closing prices.

One key thing to note from the results on the LSTM model is that the model was often somewhat accurate over a couple of days, and then started trailing off as the prediction window got larger. This can be seen in Figure 4, where the LSTM predictions are relatively accurate for the first 5 days, and then trail off. Thus, the simulation would only take into account the first couple of days of the LSTM's predictions based on the results of the model.

7 Trading Simulation

To test the theoretical performance of the model on real conditions, the model was simulated over the course of a month using data from January to February of 2023 with \$3000 to invest with. This was done through the logic defined below, which would be run daily throughout the simulation.

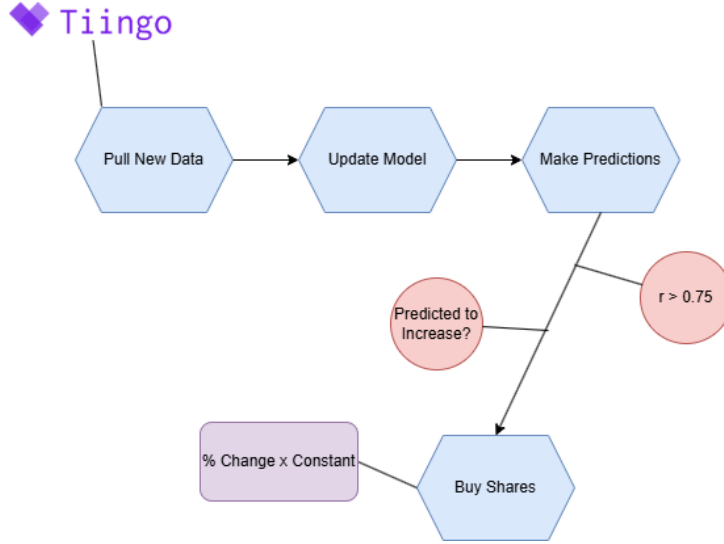


Figure 7: Flowchart of the Logic Used for the Simulation

From Figure 7, we see that the logic behind the trading is relatively simple. Every time the model made predictions, it would determine whether it should buy shares or not by seeing if the model had an r value greater than 0.75, as well as having the stock being predicted to increase. It would then buy shares based on the percent change of the closing price, along with some constant to scale the number of shares bought.

7.1 Results

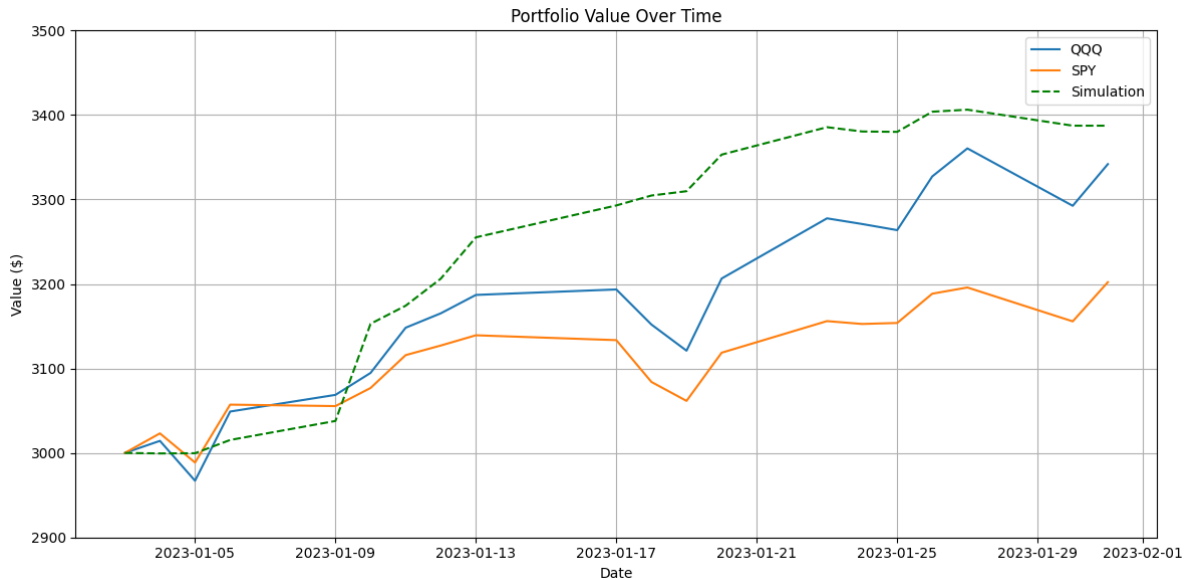


Figure 8: Simulation Results Compared to Index Funds (30 days)

Figure 8 shows the results of the simulation compared to two common index funds, the S&P 500 (SPY) and Nasdaq-100 (QQQ), with a \$3000 investment. We see that the simulation has increased its portfolio value by around 15% over the month, beating out the index funds by roughly 3% over the month. In addition to this, we see that the

simulation had relatively stable growth over the month compared to the index funds. Thus, this also shows the relative low-risk and low volatility of the models predictions and investment choices.

8 Key Outcomes and Future Directions

8.1 Key Outcomes

From this project, we see a significant outcome: ML approaches to stock trading are able to perform better than traditional methods. This was directly shown in Figure 8, where the simulation outperformed a traditional investment strategy of allocating funds to major index funds such as the S&P 500 (SPY) and Nasdaq-100 (QQQ). These results suggest that ML techniques are not just theoretical and limited to academic experiments, but can be effectively applied to real trading conditions. In addition to this, the ability of the LSTM model to be competitive against index funds over short-term market movements highlights the potential of these new models to be used for much more than long term trading strategies.

8.2 Future Directions

One significant future direction this project could be further explored in is having access to more compute power. Although the LSTM model showed in Figure 6 has an average amount of neurons and layers, carefully adding more layers with increased dropout and regularization may help model performance. However, due to compute power constraints, this option was not able to be explored in this project. With more compute power, more complex models through hyperparameter tuning could also be utilized to create custom models for different stocks.

In addition to model improvement, with more compute power, ensemble models with multiple LSTM models could also be explored. During the training of the models, it was observed that more complex models tended to overfit on the noise, while simpler models tended to generalize too much and not adapt to short term movements. Thus, an ensemble of these could help the model perform competitively in both long term and short term trading strategies.

Aside from compute power constraints, another improvement to this project could be in the data collection stage. As mentioned in section 4, data was heavily limited to OHLCV due to API call limit concerns. However, the model could potentially benefit from other features such as global economic indicators. One important note here is on the use of sentiment analysis as a feature. Although it may seem promising, news sentiment is often too late or more reactionary to market movements then the other way around, as found in Tumarkin's study [10]. Thus, model results would most likely not show any improvement with the addition of this. In addition to adding global economic indicators to the dataset, anomaly detection would also significantly improve results. Since stock data is full of noise and variability, being able to perform anomaly detection on the dataset to further clean it could help avoiding introducing the model to excess noise.

References

- [1] M. Sangeetha, “Financial Stock Market Forecast Using Evaluated Linear Regression Based Machine Learning Technique,” Elsevier.
<https://www.sciencedirect.com/science/article/pii/S2665917423002866> (accessed Feb. 23, 2025)
- [2] A. Dev, “Data Science: Understanding Random Forest machine learning model for Stock Price Prediction with Bajaj Finance Stock Data,” Medium.
<https://medium.com/@nikaljeajay36/data-science-understanding-random-forest-machine-learning-model-for-stock-price-prediction> (accessed Feb. 23, 2025)
- [3] “Long Short Term Memory Networks Explanation,” geeksforgeeks.
<https://www.geeksforgeeks.org/long-short-term-memory-networks-explanation/> (accessed Feb. 23, 2025)
- [4] B. Gulmez, “Stock price prediction with optimized deep LSTM network with artificial rabbits optimization algorithm,” Elsevier.
<https://www.sciencedirect.com/science/article/pii/S0957417423008485?via%3Dihub>
- [5] “What is Sentiment Analysis,” IBM.
<https://www.ibm.com/think/topics/sentiment-analysis> (accessed Feb. 23, 2025)
- [6] C. Palomo, “Tweet Sentiment Analysis to Predict Stock Market,” Stanford.
<https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1234/final-reports/final-report.pdf> (accessed Feb. 23, 2025)
- [7] S. Ouf, “A Deep Learning-Based LSTM for Stock Price Prediction Using Twitter Sentiment Analysis,” thesai.org.
https://thesai.org/Downloads/Volume15No12/Paper_23-A_Deep_Learning_Based_LSTM_of_Stock_Price_Prediction.pdf (accessed Feb. 23, 2024)
- [8] S. Dhanani, “Stock Prices Don’t Predict Stock Prices,” Medium.
<https://medium.com/%40shanif/stock-prices-dont-predict-stock-prices> (accessed April 26, 2025)
- [9] J. F. Jaffe, “Special Information and Insider Trading,” *The Journal of Business*, vol. 47, no. 3, pp. 410–428, 1974. <https://www.jstor.org/stable/2352458?>
- [10] R. Tumarkin, “News or Noise? Internet Postings and Stock Prices,” Taylor & Francis.
<https://www.tandfonline.com/doi/abs/10.2469/faj.v57.n3.2449>